

Desenvolvimento Web II

Persistência de Dados com SQLite

Node.js sem Framework

Professor: Prof. Me Wemerson Porto

Objetivos

Ao final desta aula você será capaz de:

- Entender o que é o SQLite
- Instalar o SQLite no Node.js
- Criar um banco de dados
- Criar tabelas
- Inserir registros
- Consultar dados
- Atualizar registros
- Remover registros
- Utilizar consultas parametrizadas
- Organizar o acesso ao banco em camadas

O problema

Imagine um sistema de biblioteca.

Livro cadastrado

↓

Servidor reinicia

↓

Todos os livros desaparecem

Guardar em variáveis não é suficiente.

Precisamos de armazenamento permanente.

Onde guardar os dados?

Podemos utilizar:

- Arquivos JSON
- Arquivos TXT
- Banco de Dados

Para aplicações pequenas e médias, existe uma excelente opção:

SQLite

O que é SQLite?

SQLite é um banco de dados relacional extremamente leve.

Diferente do MySQL ou PostgreSQL:

- ✓ Não precisa instalar um servidor
- ✓ Não precisa criar usuário
- ✓ Não precisa configurar portas
- ✓ O banco é apenas um arquivo

Como funciona?

Aplicação Node.js

|

Driver SQLite

|

biblioteca.db
(arquivo)

Todo o banco fica dentro de um único arquivo.

Vantagens

- ✓ Muito rápido
- ✓ Fácil de usar
- ✓ Gratuito
- ✓ Banco em um único arquivo
- ✓ Ideal para estudos
- ✓ Excelente para desktop e pequenos sistemas

Limitações

- ✗ Não indicado para milhares de usuários simultâneos
- ✗ Poucos recursos avançados
- ✗ Escalabilidade menor

Mesmo assim...

É excelente para aprender SQL.

Instalação

Existem dois drivers bastante utilizados.

```
sqlite3
```

ou

```
better-sqlite3
```

Nesta aula utilizaremos:

```
npm install sqlite3
```

Estrutura do projeto

```
projeto/  
package.json  
database/  
database.js  
repositories/  
produtoRepository.js  
server.js
```

Criando a conexão

```
const sqlite3 = require("sqlite3").verbose();  
const db = new sqlite3.Database("./database/biblioteca.db");
```

Se o arquivo não existir...

O SQLite cria automaticamente.

Fluxo da conexão

Node.js

↓

sqlite3

↓

Existe o arquivo?

↓

Sim → Abre

Não → Cria automaticamente

Criando uma tabela

```
CREATE TABLE livros (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  titulo TEXT,  
  autor TEXT,  
  ano INTEGER  
);
```

Executando SQL

```
db.run(`  
CREATE TABLE IF NOT EXISTS livros(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  titulo TEXT,  
  autor TEXT,  
  ano INTEGER  
)  
`);
```

Tipos de dados

SQLite possui poucos tipos.

Tipo	Exemplo
INTEGER	10
REAL	10.5
TEXT	"João"
BLOB	Arquivos
NULL	Valor vazio

Inserindo dados

SQL

```
INSERT INTO livros  
(titulo, autor, ano)  
VALUES  
( 'Node.js', 'Caio', 2025 );
```

Inserindo com Node

```
db.run(  
  `INSERT INTO livros  
  (titulo,autor,ano)  
  
  VALUES (?, ?, ?)` ,  
  ["Node.js", "Caio", 2025]  
);
```

Observe os "?"

Eles tornam a consulta segura.

SQL Injection

Nunca faça isso:

```
const sql =  
"SELECT * FROM livros  
WHERE titulo='" + titulo + "'";
```

Muito perigoso.

Consultas Parametrizadas

Sempre faça assim:

```
db.run(  
  "INSERT INTO livros  
  (titulo)  
  
  VALUES(?)",  
  
  [titulo]  
);
```

Muito mais seguro.

Consultando dados

```
db.all(  
  "SELECT * FROM livros",  
  [],  
  (err, rows)=>{  
    console.log(rows);  
  }  
);
```

Resultado

```
[  
  {  
    id:1,  
    titulo:"Node.js",  
    autor:"Caio"  
  }  
]
```

O SQLite retorna um Array de Objetos.

Buscando apenas um registro

```
db.get(  
  "SELECT * FROM livros  
  WHERE id=?",  
  [1],  
  (err, row) => {  
    console.log(row);  
  }  
);
```

Atualizando dados

SQL

```
UPDATE livros  
SET titulo='Node Avançado'  
WHERE id=1;
```

Atualizando com Node

```
db.run(  
  `UPDATE livros  
  SET titulo=?  
  WHERE id=?`,  
  ["Node Avançado", 1]  
);
```

Excluindo registros

```
db.run(  
  
  "DELETE FROM livros  
  WHERE id=?",  
  
  [1]  
  
);
```

CRUD

Create



Read



Update



Delete

Todo sistema administrativo realiza essas operações.

Criando um Repository

Controller

↓

Repository

↓

SQLite

O Repository concentra todas as consultas SQL.

Exemplo

```
const db =  
require("../database/database");  
  
function listar(){  
  db.all(  
    "SELECT * FROM livros",  
    [],  
    (err, rows)=>{  
      console.log(rows);  
    });  
}
```

Inserindo um livro

```
function inserir(livro){  
  db.run(  
    `INSERT INTO livros  
    (titulo,autor,ano)  
    VALUES (?, ?, ?)` ,  
    [  
      livro.titulo,  
      livro.autor,  
      livro.ano  
    ]  
  );  
}
```

Buscando por ID

```
function buscar(id){  
  db.get(  
    "SELECT * FROM livros  
    WHERE id=?",  
    [id],  
    (err,row)=>{  
      console.log(row);  
    });  
}
```

Atualizando

```
function atualizar(id, livro){  
  db.run(  
    `UPDATE livros  
  
    SET titulo=?,  
    autor=?,  
    ano=?  
  
    WHERE id=?`,  
    [  
      livro.titulo,  
      livro.autor,  
      livro.ano,  
      id  
    ]  
  );  
}
```

Removendo

```
function remover(id){  
  db.run(  
    "DELETE FROM livros  
    WHERE id=?",  
    [id]  
  );  
}
```

Tratando erros

```
db.run(  
  sql,  
  parametros,  
  function(err){  
    if(err){  
      console.log(err);  
    }  
  }  
);
```

Nunca ignore erros.

Organização

Projeto



Vantagens dessa organização

- ✓ Código organizado
- ✓ Reutilização
- ✓ Fácil manutenção
- ✓ Menos repetição
- ✓ Separação de responsabilidades

Fluxo completo

Usuário



Node.js



Repository



SQLite



Arquivo biblioteca.db



Resposta

Boas práticas

- ✓ Use IF NOT EXISTS
- ✓ Utilize consultas parametrizadas
- ✓ Centralize a conexão
- ✓ Separe Repository
- ✓ Nomeie bem tabelas
- ✓ Trate erros

O que evitar

- ✗ SQL concatenado
- ✗ Criar várias conexões
- ✗ SQL espalhado pelo projeto
- ✗ Ignorar erros
- ✗ Misturar regras de negócio com SQL

Comparação

SQLite	MySQL
Arquivo único	Servidor
Sem instalação	Instalação necessária
Ideal para estudos	Ideal para produção
Muito simples	Mais robusto
Pouca configuração	Maior configuração

Quando usar SQLite?

Excelente para:

- Projetos acadêmicos
- Sistemas locais
- Aplicações desktop
- Protótipos
- APIs pequenas
- Aprender SQL

Resumo

Hoje aprendemos:

- O que é SQLite
- Como criar um banco
- Criar tabelas
- Inserir dados
- Consultar registros
- Atualizar
- Excluir
- Consultas parametrizadas
- Organização em Repository

Exercício Prático

Desenvolva um sistema de cadastro de produtos.

Tabela: produtos

```
id  
nome  
preco  
estoque
```

Implemente:

- Criar tabela automaticamente
- Inserir produto
- Listar produtos
- Buscar por ID
- ...

Exercício Prático

Implemente:

- Atualizar produto
- Excluir produto

Desafio extra: implemente um menu interativo no terminal usando `readline`, permitindo que o usuário realize todas as operações do CRUD sem alterar o código-fonte.

Utilize:

- `sqlite3`
- consultas parametrizadas